Lecture 4: Logistic Regression

Course Teacher: Md. Shariful Islam Bhuyan

Learning Journey

1. From Regression to Classification
2. Linear Model for Classification
3. The Softmax Function
4. Cross-Entropy Loss
5. Information Theory Perspective
6. Implementation
7. Generalization in Classification
8. Distribution Shift
9. Hands-On Exercise
10. Summary

The Big Picture: What Changes?

Think About It

You've mastered predicting **continuous** values (house prices).
But what if you need to answer "**Which category**?" instead of "**How much**?"

Regression (Chapter 3):

- Output: continuous value
- "Price = $425,000"
- "Temperature = 23.5℃"
- Loss: Mean Squared Error

Classification (This Chapter):

- Output: discrete category
- "This is a cat"
- "Email is spam"
- Loss: Cross-Entropy

Key Insight

The "plumbing" remains the same: load data → forward pass → compute loss → backprop → update.
What changes: **output layer**, **loss function**, and **target representation**.

Hard vs. Soft Classification

Two subtly different problems:

1. **Hard Assignment**: Just tell me the category!
   - "Is this a dog or a cat?" $\rightarrow$ "It's a dog."
2. **Soft Assignment**: How confident are you?
   - "Is this a dog or a cat?" $\rightarrow$ "70% dog, 30% cat"

Think About It

Why would we want probabilities instead of just the answer?

Hard vs. Soft Classification

Two subtly different problems:

1. **Hard Assignment**: Just tell me the category!
   - "Is this a dog or a cat?" → "It's a dog."
2. **Soft Assignment**: How confident are you?
   - "Is this a dog or a cat?" → "70% dog, 30% cat"

Think About It

Why would we want probabilities instead of just the answer?

Reasons for soft predictions:

- **Uncertainty quantification**: "I'm not sure" is valuable information
- **Decision thresholds**: Medical diagnosis—maybe 70% isn't enough
- **Downstream processing**: Combine with other models
- **Training signal**: Richer gradients for learning

Representing Labels: One-Hot Encoding

Problem: How do we represent "cat", "dog", "chicken" for a neural network?

Naive approach: $y \in \{1, 2, 3\}$

- 1 = dog, 2 = cat, 3 = chicken

Watch Out!

This implies ordering!
Is dog < cat < chicken?
Is |dog - chicken| = 2?

Better: One-Hot Encoding

$$y \in \left\{ \underbrace{\begin{pmatrix}1\\0\\0\end{pmatrix}}_{\text{cat}}, \underbrace{\begin{pmatrix}0\\1\\0\end{pmatrix}}_{\text{chicken}}, \underbrace{\begin{pmatrix}0\\0\\1\end{pmatrix}}_{\text{dog}} \right\}$$

- All classes are **equidistant**
- No implicit ordering
- Works perfectly with probability outputs

Key Insight

One-hot vectors have q components for q classes.
Only the true class has value 1; all others are 0.

Setting Up the Problem

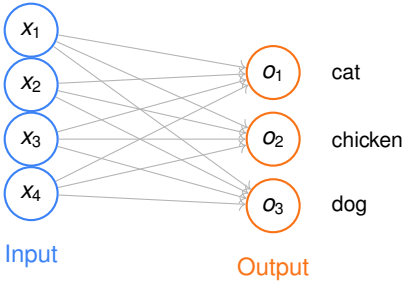
Concrete Example: Classify 2×2 grayscale images into 3 categories

Input: 4 features

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix}$$

(flattened 2×2 image)

Output: 3 classes
(cat, chicken, dog)



Think About It

How many parameters do we need?
(Each output depends on every input—this is a **fully connected** layer)

The Linear Model: One Output Per Class

Key Idea: Compute one **affine function** for each class

Detailed equations (for 4 features, 3 classes):

$$
\begin{aligned}
o_1 &= x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14} + b_1 \\
o_2 &= x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24} + b_2 \\
o_3 &= x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34} + b_3
\end{aligned}
$$

Parameter count:

- Weights: $4 \times 3 = 12$ scalars (matrix $\mathbf{W} \in \mathbb{R}^{4\times 3}$)
- Biases: 3 scalars (vector $\mathbf{b} \in \mathbb{R}^3$)
- Total: 15 parameters

Compact matrix notation:

$$\boxed{\mathbf{o} = \mathbf{W}^\top \mathbf{x} + \mathbf{b}}$$

where $\mathbf{W} \in \mathbb{R}^{d\times q}$ for d features and q classes.

Why Can't We Just Use These Outputs Directly?

Think About It

We have outputs o_1, o_2, o_3 for each class.
Why not just pick the largest one and call it a day?

For prediction: Yes! $\text{argmax}_j\, o_j$ gives the predicted class.
For training: Two critical problems:

Watch Out!

Problem 1: No guarantee $\sum_i o_i = 1$

Outputs could be: $(5.2, -3.1, 8.7)$
Sum = $10.8 \neq 1$

Watch Out!

Problem 2: o_i can be negative!

We want probabilities: $0 \leq \hat{y}_i \leq 1$
Can't have $P(\text{cat}) = -3.1$

Key Insight

Need a function that transforms arbitrary real numbers into valid probabilities: **The Softmax Function**

Designing Softmax: First Principles

Goal: Transform $\mathbf{o} = (o_1, \ldots, o_q)$ into probabilities $\hat{\mathbf{y}} = (\hat{y}_1, \ldots, \hat{y}_q)$

Requirements:

1. $\hat{y}_i \geq 0$ for all i (non-negative)
2. $\sum_{i=1}^{q} \hat{y}_i = 1$ (sum to one)
3. Larger $o_i \Rightarrow$ larger $\hat{y}_i$ (preserve ordering)
4. Differentiable (for gradient descent)

Think About It

What function is always positive, monotonically increasing, and differentiable?

Designing Softmax: First Principles

Goal: Transform $\mathbf{o} = (o_1, \ldots, o_q)$ into probabilities $\hat{\mathbf{y}} = (\hat{y}_1, \ldots, \hat{y}_q)$

Requirements:

1. $\hat{y}_i \geq 0$ for all i (non-negative)
2. $\sum_{i=1}^{q} \hat{y}_i = 1$ (sum to one)
3. Larger $o_i \Rightarrow$ larger $\hat{y}_i$ (preserve ordering)
4. Differentiable (for gradient descent)

Think About It

What function is always positive, monotonically increasing, and differentiable?

The exponential function! $e^x > 0$ for all $x \in \mathbb{R}$

Two-step construction:

1. Make positive: $\exp(o_i)$ — ensures non-negativity
2. Normalize: divide by sum — ensures they add to 1

The Softmax Formula

Softmax Function

$$\boxed{\hat{y}_i = \text{softmax}(\mathbf{o})_i = \frac{\exp(o_i)}{\sum_{k=1}^{q} \exp(o_k)}}$$

Step-by-step example: Let $\mathbf{o} = (2, 1, 0.1)$

1. **Exponentiate:** $(\exp(2), \exp(1), \exp(0.1)) = (7.39, 2.72, 1.11)$
2. **Sum:** $7.39 + 2.72 + 1.11 = 11.22$
3. **Normalize:** $\hat{\mathbf{y}} = \left(\frac{7.39}{11.22}, \frac{2.72}{11.22}, \frac{1.11}{11.22}\right) = (0.66, 0.24, 0.10)$

Verify: $0.66 + 0.24 + 0.10 = 1.0$ ✓

Key Insight

The predicted class is: $\text{argmax}_j \hat{y}_j = \text{argmax}_j o_j$
Softmax preserves the ordering! We only need it for probabilities and training.

Softmax: Intuition from Physics

Historical Connection: Boltzmann Distribution

In statistical physics, the probability of a system being in state i with energy E_i:

$$P(i) \propto \exp\left(-\frac{E_i}{kT}\right)$$

where k is Boltzmann's constant and T is temperature.

In our context: Think of o_i as negative energy (higher o_i = lower energy = more probable)

Temperature scaling: $\mathrm{softmax}(\mathbf{o}/T)$

High T (hot):

- Probabilities $\rightarrow$ uniform
- More exploration
- "Uncertain"

$T = 1$ (default):

- Standard softmax
- Balanced

Low T (cold):

- Winner-take-all
- $T \rightarrow 0$: hard max
- "Confident"

Why "Soft" Max?

The name comes from being a "soft" version of the max function:

RealSoftMax (Log-Sum-Exp)

For two numbers:

$$\text{RealSoftMax}(a, b) = \log(\exp(a) + \exp(b))$$

Key property: $\text{RealSoftMax}(a, b) > \max(a, b)$

Proof: Let $a \geq b$ (WLOG). Then:

$$\begin{aligned}
\text{RealSoftMax}(a, b) &= \log(\exp(a) + \exp(b)) \\
&= \log\left(\exp(a)\left(1 + \exp(b - a)\right)\right) \\
&= a + \log(1 + \exp(b - a)) \\
&> a = \max(a, b) \quad \text{since } \log(1 + x) > 0 \text{ for } x > 0
\end{aligned}$$

As $\lambda \to \infty$: $\lambda^{-1}\text{RealSoftMax}(\lambda a, \lambda b) \to \max(a, b)$

The Training Objective: Maximum Likelihood

Think About It

We have predictions $\hat{\mathbf{y}}$ and true labels $\mathbf{y}$ (one-hot).
What loss function should we minimize?

Maximum Likelihood Principle:
Given data $\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^{n}$, find parameters that maximize:

$$P(\mathbf{Y}|\mathbf{X}) = \prod_{i=1}^{n} P(\mathbf{y}^{(i)}|\mathbf{x}^{(i)})$$

Taking the negative log (for numerical stability and to convert max $\rightarrow$ min):

$$-\log P(\mathbf{Y}|\mathbf{X}) = -\sum_{i=1}^{n} \log P(\mathbf{y}^{(i)}|\mathbf{x}^{(i)}) = \sum_{i=1}^{n} \ell(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)})$$

This gives us the **negative log-likelihood** loss!

Deriving Cross-Entropy Loss

Setup: True label $\mathbf{y}$ is one-hot, prediction $\hat{\mathbf{y}}$ is softmax output

What is $P(\mathbf{y}|\mathbf{x})$?
If true class is c (so $y_c = 1$, all others 0), then:

$$P(\mathbf{y}|\mathbf{x}) = \hat{y}_c = \prod_{j=1}^{q} \hat{y}_j^{y_j}$$

(Only the c-th term survives since $y_j = 0$ for $j \neq c$)

Taking negative log:

$$\ell(\mathbf{y}, \hat{\mathbf{y}}) = -\log P(\mathbf{y}|\mathbf{x}) = -\log \prod_{j=1}^{q} \hat{y}_j^{y_j}$$

$$= -\sum_{j=1}^{q} y_j \log \hat{y}_j$$

Cross-Entropy Loss

q

Cross-Entropy: Simplified for One-Hot Labels

Since $\mathbf{y}$ is one-hot (only one $y_j = 1$), the loss simplifies:

$$\ell(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^{q} y_j \log \hat{y}_j = -\log \hat{y}_c$$

where c is the true class index.

Intuition: We're penalizing the negative log of the predicted probability for the *correct* class.

If $\hat{y}_c \approx 1$ (correct, confident):

$$\ell = -\log(1) = 0$$

$\Rightarrow$ No penalty!

If $\hat{y}_c \approx 0$ (wrong, confident):

$$\ell = -\log(0^+) = +\infty$$

$\Rightarrow$ Huge penalty!

Key Insight

Cross-entropy strongly penalizes confident wrong predictions.
This is exactly what we want for classification!

Combining Softmax and Cross-Entropy

Plugging softmax into cross-entropy:

$$\ell(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^{q} y_j \log \frac{\exp(o_j)}{\sum_k \exp(o_k)}$$

$$= -\sum_{j=1}^{q} y_j \left[o_j - \log \sum_{k=1}^{q} \exp(o_k) \right]$$

$$= -\sum_{j=1}^{q} y_j \cdot o_j + \underbrace{\sum_{j=1}^{q} y_j}_{=1} \cdot \log \sum_{k=1}^{q} \exp(o_k)$$

$$= \log \sum_{k=1}^{q} \exp(o_k) - \sum_{j=1}^{q} y_j \cdot o_j$$

Combined Softmax Cross-Entropy Loss

The Beautiful Gradient

Compute the gradient: $\frac{\partial \ell}{\partial o_j}$

$$\frac{\partial \ell}{\partial o_j} = \frac{\partial}{\partial o_j}\left[\log \sum_k \exp(o_k) - \sum_k y_k \cdot o_k\right]$$

$$= \frac{\exp(o_j)}{\sum_k \exp(o_k)} - y_j$$

$$= \text{softmax}(\mathbf{o})_j - y_j$$

$$= \hat{y}_j - y_j$$

Key Insight

The gradient is simply: (prediction) − (target)

$$\boxed{\nabla_{\mathbf{o}} \ell = \hat{\mathbf{y}} - \mathbf{y}}$$

Why “Cross-Entropy”?

Think About It

The name “cross-entropy” comes from information theory.
What does it mean intuitively?

Shannon’s Entropy: For distribution P:

$$H[P] = -\sum_j P(j) \log P(j) = \mathbb{E}_{j\sim P}[-\log P(j)]$$

Interpretation: Average “surprise” (bits needed) to encode outcomes from P.

Cross-Entropy: Between true distribution P and model Q:

$$H(P, Q) = -\sum_j P(j) \log Q(j) = \mathbb{E}_{j\sim P}[-\log Q(j)]$$

Interpretation: Average bits needed to encode outcomes from P using a code designed for Q.

Key Insight

Surprisal: The Building Block

Claude Shannon's Key Insight:

The "surprise" of observing event j with probability $P(j)$:

$$\text{Surprisal}(j) = -\log P(j) = \log \frac{1}{P(j)}$$

High probability event:

- $P(j) = 0.99$
- Surprisal $= -\log(0.99) \approx 0.01$
- "Not surprising at all"

Low probability event:

- $P(j) = 0.01$
- Surprisal $= -\log(0.01) \approx 4.6$
- "Very surprising!"

Connection to compression:

- Easy to predict $\Rightarrow$ easy to compress
- Surprisal $\approx$ bits needed to encode the event

Two Views of the Same Objective

We can interpret cross-entropy loss in two equivalent ways:

View 1: Maximum Likelihood

Maximize:

$$\prod_i P(\mathbf{y}^{(i)}|\mathbf{x}^{(i)};\theta)$$

Equivalent to minimizing negative log-likelihood.

"Find parameters that make the observed data most probable."

View 2: Minimize Surprisal

Minimize:

$$\mathbb{E}[-\log \hat{y}_{\text{true}}]$$

Minimize average surprise when seeing true labels.

"Make the true labels as unsurprising as possible."

Key Insight

Both views lead to the same optimization!
This deep connection between probability and information theory is fundamental.

The Fashion-MNIST Dataset

Our playground: 10 categories of clothing images

Dataset specs:

- 28×28 grayscale images
- 60,000 training / 10,000 test
- 10 classes:
  - T-shirt, Trouser, Pullover
  - Dress, Coat, Sandal
  - Shirt, Sneaker, Bag
  - Ankle boot

Why Fashion-MNIST?

- More challenging than MNIST
- Quick to train
- Good for prototyping

```python
import torch
import torchvision
from torchvision import transforms

# Load dataset
trans = transforms.Compose([
    transforms.ToTensor()
])

train = torchvision.datasets.FashionMNIST(
    root='./data', train=True,
    transform=trans, download=True)

# Check shape
print(train[0][0].shape)
# torch.Size([1, 28, 28])
```

Implementing Softmax from Scratch

```python
def softmax(X):
    """Numerically stable softmax"""
    # Subtract max for stability
    X_max = X.max(dim=1,
                  keepdim=True).values
    X_exp = torch.exp(X - X_max)
    partition = X_exp.sum(dim=1,
                          keepdim=True)
    return X_exp / partition

# Test
X = torch.tensor([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0]])
probs = softmax(X)
print(probs.sum(dim=1))
# tensor([1., 1.])
```

Watch Out!

Numerical Stability!

Naive: $\exp(1000) = \infty$ (overflow)

Solution: Subtract max first!

$$\frac{\exp(o_j)}{\sum_k \exp(o_k)} = \frac{\exp(o_j - \bar{o})}{\sum_k \exp(o_k - \bar{o})}$$

where $\bar{o} = \max_k o_k$

Now $o_j - \bar{o} \leq 0$ for all j, so exp won't overflow!

Cross-Entropy Loss Implementation

From scratch:

```python
def cross_entropy(y_hat, y):
    """Cross-entropy loss
    y_hat: (batch, num_classes)
    y: (batch,) - class indices
    """
    # Select probs for true class
    return -torch.log(
        y_hat[range(len(y_hat)), y]
    ).mean()

# Example
y_hat = torch.tensor(
    [[0.1, 0.3, 0.6],
     [0.3, 0.2, 0.5]])
y = torch.tensor([0, 2])
print(cross_entropy(y_hat, y))
# tensor(1.4979)
```

Using PyTorch (preferred):

```python
import torch.nn.functional as F

# Takes logits directly!
# (combines softmax + cross-entropy)
loss = F.cross_entropy(logits, y)
```

Key Insight

PyTorch's `cross_entropy` takes **logits** (raw scores), not probabilities!

It uses the "LogSumExp trick" for numerical stability:

$$\log \hat{y}_j = o_j - \log \sum_k e^{o_k}$$

The Complete Model

```python
import torch
from torch import nn

class SoftmaxRegression(nn.Module):
    def __init__(self, num_inputs, num_outputs):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),            # (batch, 1, 28, 28) -> (batch, 784)
            nn.Linear(num_inputs, num_outputs)  # 784 -> 10
        )

    def forward(self, X):
        return self.net(X)  # Return logits, not softmax!

# Create model
model = SoftmaxRegression(num_inputs=784, num_outputs=10)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

# Training loop (one step)
logits = model(X_batch)
loss = loss_fn(logits, y_batch)
loss.backward()
optimizer.step()
optimizer.zero_grad()
```

The Test Set: Our Reality Check

Think About It

Training accuracy is 99%. Is our model good?
(Hint: Remember overfitting from Chapter 3!)

Key quantities:

- **Empirical error** (on test set $\mathcal{D}$):

$$\epsilon_{\mathcal{D}}(f) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}(f(\mathbf{x}^{(i)}) \neq y^{(i)})$$

- **Population error** (true error):

$$\epsilon(f) = \mathbb{E}_{(\mathbf{x},y)\sim P}[\mathbf{1}(f(\mathbf{x}) \neq y)]$$

Key Insight

We can only *estimate* the true error using test data.
Central Limit Theorem: error converges at rate $O(1/\sqrt{n})$.

How Many Test Samples Do We Need?

Hoeffding's Inequality:

$$P\left(|\epsilon_{\mathcal{D}}(f) - \epsilon(f)| \geq t\right) < 2\exp(-2nt^2)$$

Example: Want 95% confidence that test error is within $\pm 1\%$ of true error.

- Set $2\exp(-2nt^2) = 0.05$ with $t = 0.01$
- Solve: $n \geq \frac{\ln(40)}{2 \times 0.01^2} \approx 18,444$

Watch Out!

This is a lot! Popular benchmarks (MNIST, Fashion-MNIST) have 10,000 test samples.
Claims of 0.1% improvements may not be statistically significant!

Rule of thumb: $n \approx 10,000$ gives $\pm 1\%$ at 95% confidence.

The Test Set Reuse Problem

Think About It

You train model f_1, test it, get 85% accuracy.
You train model f_2, test it, get 87% accuracy.
Is f_2 really better?

The Test Set Reuse Problem

Think About It

You train model f_1, test it, get 85% accuracy.
You train model f_2, test it, get 87% accuracy.
Is f_2 really better?

Problem 1: Multiple Hypothesis Testing

- With 20 models, probability that *at least one* has misleading test error is high
- "p-hacking" in ML: keep trying until something looks good

Problem 2: Adaptive Overfitting

- f_2 was designed *after* seeing f_1's test performance
- Information has "leaked" from test set to model design
- Test set is no longer truly independent!

Key Insight

Best practice: Use validation set for model selection, reserve test set for final evaluation only!

When Training $\neq$ Deployment

The fundamental assumption we've made:

$$(\mathbf{x}_{\text{train}}, y_{\text{train}}) \sim P(\mathbf{x}, y) \quad \text{and} \quad (\mathbf{x}_{\text{test}}, y_{\text{test}}) \sim P(\mathbf{x}, y)$$

Same distribution!

Watch Out!

In practice, this assumption often fails!

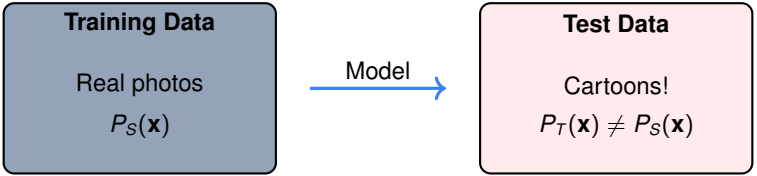
Training data: $P_S(\mathbf{x}, y)$ (source)
Test/deployment data: $P_T(\mathbf{x}, y)$ (target)

Three types of distribution shift:

1. **Covariate shift:** $P_S(\mathbf{x}) \neq P_T(\mathbf{x})$, but $P(y|\mathbf{x})$ unchanged
2. **Label shift:** $P_S(y) \neq P_T(y)$, but $P(\mathbf{x}|y)$ unchanged
3. **Concept shift:** $P(y|\mathbf{x})$ itself changes!

Covariate Shift: Example

Scenario: Train cat/dog classifier on photos, deploy on cartoons



Key assumption for correction: $P(y|\mathbf{x})$ is the same!

- A cat is still a cat, whether photo or cartoon
- $P_S(y|\mathbf{x}) = P_T(y|\mathbf{x})$

Correction strategy: Reweight training samples by $\frac{P_T(\mathbf{x})}{P_S(\mathbf{x})}$

Label Shift: Medical Diagnosis Example

Scenario: Disease prevalence changes seasonally

Training (summer):

- Flu cases: 10%
- Allergies: 60%
- Other: 30%

$P_S(y)$: label distribution

Deployment (winter):

- Flu cases: 40%
- Allergies: 20%
- Other: 40%

$P_T(y) \neq P_S(y)$

Key assumption: $P(\mathbf{x}|y)$ unchanged (symptoms given disease are stable)

Key Insight

Label shift is natural when y *causes* $\mathbf{x}$.
(Disease causes symptoms, not the other way around!)

Concept Shift: When Definitions Change

The trickiest case: $P(y|\mathbf{x})$ itself changes!

Examples:

- **Fashion:** What counts as "stylish" changes over time
- **Spam detection:** Spammers adapt to filters
- **Product recommendations:** Seasonal preferences (Santa hats in July?)
- **Language:** Word meanings evolve

Watch Out!

Concept shift is hard to fix!

Often requires:

- Collecting new labeled data
- Continuous model updates
- Human-in-the-loop systems

Exercise: Build Your Own Classifier

Task: Implement softmax regression on Fashion-MNIST

```
# Step 1: Load data
from torchvision import datasets, transforms
train_data = datasets.FashionMNIST('./data', train=True, download=True,
                                   transform=transforms.ToTensor())
train_loader = torch.utils.data.DataLoader(train_data, batch_size=256, shuffle=True)

# Step 2: Define model (TODO: Fill in)
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(???, ???)  # What dimensions?
)

# Step 3: Training loop
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
for epoch in range(10):
    for X, y in train_loader:
        logits = model(X)
        loss = F.cross_entropy(logits, y)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
```

Exercise Questions

1. **Dimensions:** What should replace the `???` in `nn.Linear(???, ???)`?
   - Input: $28 \times 28 =$ _____ features
   - Output: _____ classes

2. **Experiment:** What happens if you:
   - Increase learning rate to 1.0?
   - Decrease batch size to 32?
   - Train for 100 epochs?

3. **Analysis:** Look at misclassified images. What patterns do you see?
   - Which classes get confused with each other?
   - Why might this happen?

4. **Challenge:** Can you achieve $> 85\%$ test accuracy with just a linear model?

Key Takeaways

Classification Fundamentals:

- One-hot encoding for labels
- Softmax converts scores to probabilities
- Cross-entropy measures prediction quality
- Gradient = (prediction - target)

Information Theory View:

- Entropy = average surprise
- Cross-entropy = expected coding cost
- MLE $\Leftrightarrow$ minimize cross-entropy

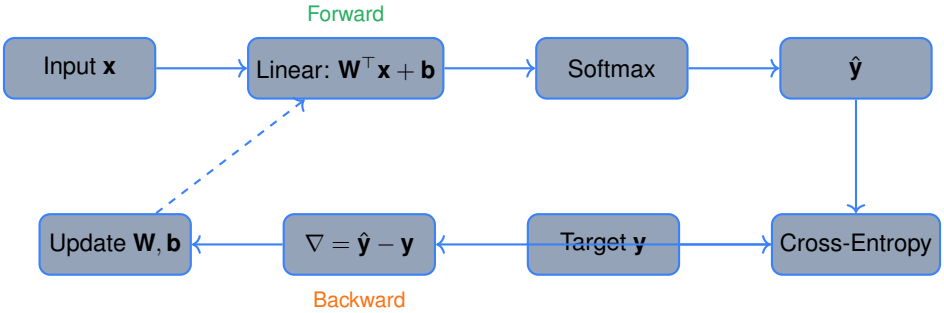
Practical Considerations:

- Numerical stability matters!
- Use framework's combined functions
- Test set $\neq$ validation set
- Watch for distribution shift

Coming Next:

- Chapter 5: Multilayer Perceptrons
- Adding hidden layers
- Non-linear activations
- The "deep" in deep learning!

The Complete Pipeline



Thank You!

Full book: `https://d2l.ai`